

Fly-in Drones - Translation

フライインドローンは面白い。

Fly-in Drones are interesting.

概要：複数のドローンを接続されたゾーンを通じてナビゲートし、シミュレーションのターンを最小限に抑えつつ、移動の制約を処理する効率的なドローンルーティングシステムを設計する。

Summary: Design an efficient drone routing system that navigates multiple drones through connected zones while minimizing simulation turns and handling movement constraints.

バージョン：1.6

Version: 1.6

目次

Contents

I まえがき	2
II AIの指示	3
III 共通の指示	5
III.1 一般規則	5
III.2 Makefile	5
III.3 追加ガイドライン	6
IV はじめに	7
V 制約事項	8
VI ドローンを飛ばす	9
VII 必須パート	11
VII.1 経路探索とアルゴリズムの要件	11
VII.2 ゾーンの占有規則	12
VII.3 移動とターンの仕組み	12
VII.4 パーサーの制約	13
VII.5 シミュレーション出力形式	15
VII.6 スコアリングシステム	15
VII.7 パフォーマンスベンチマーク	17
VIII Readmeの要件	20
IX ボーナスパート	21
X 提出とピアレビュー	22

I Foreword	2
II AI Instructions	3
III Common Instructions	5
III.1 General Rules	5
III.2 Makefile	5
III.3 Additional Guidelines	6
IV Introduction	7
V Constraints	8
VI Let the drone fly	9
VII Mandatory Part	11
VII.1 Pathfinding and Algorithm Requirements	11
VII.2 Zone Occupancy Rules	12
VII.3 Movement and Turn Mechanics	12
VII.4 Parser Constraints	13
VII.5 Simulation Output Format	15
VII.6 Scoring System	15
VII.7 Performance Benchmarks	17
VIII Readme Requirements	20
IX Bonus Part	21
X Submission and peer-review	22

ドローンはニュージーランドで羊の群れを誘導するために使用されており、ブンブンと音を立てる空中の羊飼いが牧羊犬に取って代わっています。日本では、オフィスビルで大音量の音楽を流し、ライトを点滅させて、働きすぎの従業員を物理的に家に追い返すドローンが配備されています。あるドローンは、飛行中に壁にグラフィティを描くように訓練されており、テクノロジーとストリートアートの反抗的な融合となっています。スウェーデンでは、科学者たちが絶滅危惧種を研究するために、海に浮かぶクジラのフンを嗅ぎつけるドローンを使用しました。鳥や昆虫の形をした実験的なドローンもあり、羽ばたきなどを模倣して気付かれずにスパイ活動を行います。プロペラを全く使用せず、シャボン玉を羽ばたかせて飛行するドローンさえあります。火山研究では、かつてドローンが噴火雲に直接飛び込み、空中で溶けましたが、崩壊する数秒前にデータを送り返すことに成功しました。そして韓国では、同調したドローンショーが花火に取って代わりました。より安全で、静かで、そしてどういうわけかさらに魔法のようです。

Drones have been used to herd sheep in New Zealand, replacing sheepdogs with buzzing aerial shepherds. In Japan, office buildings deploy drones that play loud music and flash lights to literally chase overworked employees home. One drone was trained to paint graffiti on walls mid-flight a rebellious blend of tech and street art. In Sweden, scientists used drones to sniff out whale poop floating on the ocean to study endangered species. Some experimental drones are shaped like birds or insects to spy without being noticed, flapping wings and all. There's even a drone that flies by flapping soap bubbles, no propellers involved. In volcano research, a drone once flew straight into an eruption cloud, melted mid-air, but managed to send back data just seconds before disintegration. And in South Korea, synchronized drone shows have replaced fireworks safer, silent, and somehow even more magical.

「車輪の再発明」というフレーズは、車輪が古くから存在し、非常にうまく機能している素晴らしい発明であるという考えから来ています。それに何の問題もないので、それを再び発明しようとすることは実際には役に立たず、時間の無駄になる可能性があります。特に、その時間を代わりに新しい問題の解決に費やすことができる場合はそうです。

The phrase comes from the idea that the wheel is a brilliant invention that's been around forever and works really well. Since there's nothing wrong with it, trying to invent it again wouldn't really help and could be a waste of time especially when that time could be spent solving new problems instead.

プログラミングにおいて、これは、すでに存在しているものをゼロから構築しようとする人に起こります。確固たるオープンソースのバージョンがすでに世に出ているのに、独自の並べ替えアルゴリズムやフレームワークを記述するようなものです。しかし、それがすべて悪いわけではありません。自分で構築することは、内部で物事がどのように機能するかを学ぶ素晴らしい方法になり得ます。重要なのはバランスを見つけることです。すべてを再構築するのではなく、使用しているツールが実際にどのように機能するかを探求する時間を作ることです。そうすることで、同じ古い車輪を再発明することに行き詰まることなく、開発者として成長することができます。

In programming, this happens when someone builds something from scratch that already exists like writing your own sorting algorithm or framework when solid, open-source versions are already out there. But it's not all bad: doing it yourself can be a great way to learn how things work under the hood. The key is to find a balance - don't rebuild everything, but take time to explore how the tools you're using actually work. That way, you'll grow as a developer without getting stuck reinventing the same old wheels.

第2章

AIの指示

Chapter II
AI Instructions

コンテキスト

Context

学習の過程で、AIは様々なタスクを支援することができます。AIツールの様々な機能を探索し、それらがどのようにあなたの作業をサポートできるかを理解するために時間をかけてください。ただし、常に慎重にアプローチし、結果を批判的に評価してください。コード、ドキュメント、アイデア、または技術的な説明であっても、自分の質問が適切に構成されていたか、生成されたコンテンツが正確であるかを完全に確信することはできません。あなたの仲間は、間違いや死角を避けるための貴重なリソースです。

During your learning journey, AI can assist with many different tasks. Take the time to explore the various capabilities of AI tools and how they can support your work. However, always approach them with caution and critically assess the results. Whether it's code, documentation, ideas, or technical explanations, you can never be completely sure that your question was well-formed or that the generated content is accurate. Your peers are a valuable resource to help you avoid mistakes and blind spots.

主なメッセージ

Main message

反復的または退屈なタスクを減らすためにAIを使用してください。

将来のキャリアに役立つプロンプトスキル（コーディングと非コーディングの両方）を開発してください。

一般的なリスク、バイアス、倫理的課題をよりよく予測し、回避するために、AIシステムがどのように機能するかを学んでください。

仲間と協力することで、技術的スキルとヒューマンスキルの両方を構築し続けてください。

完全に理解し、責任を持つことができるAI生成コンテンツのみを使用してください。

Use AI to reduce repetitive or tedious tasks.

Develop prompting skills both coding and non-coding that will benefit your future career.

Learn how AI systems work to better anticipate and avoid common risks, biases, and ethical issues.

Continue building both technical and power skills by working with your peers.

Only use AI-generated content that you fully understand and can take responsibility for.

• 学習者の規則：

- AIツールを倫理的に使用し、潜在的なバイアスを減らすことができるように、AIツールを探索し、それらがどのように機能するかを理解するために時間をかける必要があります。
- プロンプトを出す前に問題について熟考する必要があります。これにより、正確な語彙を使用して、より明確で詳細かつ関連性の高いプロンプトを作成することができます。
- AIによって生成されたものを体系的にチェック、レビュー、質問、テストする習慣を身につける必要があります。
- 常にピアレビューを求める必要があります。自分自身の検証だけに頼ってはいけません。

• Learner rules:

- You should take the time to explore AI tools and understand how they work, so you can use them ethically and reduce potential biases.
- You should reflect on your problem before prompting this helps you write clearer, more detailed, and more relevant prompts using accurate vocabulary.
- You should develop the habit of systematically checking, reviewing, questioning, and testing anything generated by AI.
- You should always seek peer review don't rely solely on your own validation.

フェーズの成果：

Phase outcomes:

- 汎用およびドメイン固有のプロンプトスキルの両方を開発する。
- AIツールの効果的な使用により生産性を向上させる。
- 計算論的思考、問題解決、適応性、およびコラボレーションを強化し続ける。

- Develop both general-purpose and domain-specific prompting skills.
- Boost your productivity with effective use of AI tools.
- Continue strengthening computational thinking, problem-solving, adaptability, and collaboration.

• コメントと例：

• Comments and examples:

- 試験、評価など、真の理解を示す必要がある状況に定期的に遭遇します。準備を整え、技術的スキルと対人スキルの両方を構築し続けてください。
- 自分の推論を説明し、仲間と議論することで、多くの場合、理解のギャップが明らかになります。ピアラーニングを優先してください。
- AIツールには、あなたの特定のコンテキストが欠けていることが多く、一般的な応答を提供する傾向があります。あなたの環境を共有する仲間は、より関連性が高く正確な洞察を提供することができます。
- AIが最も可能性の高い答えを生成する傾向があるのに対し、仲間は代替の視点や貴重なニュアンスを提供することができます。品質チェックポイントとして彼らに頼ってください。

- You'll regularly encounter situations exams, evaluations, and more where you must demonstrate real understanding. Be prepared, keep building both your technical and interpersonal skills.
- Explaining your reasoning and debating with peers often reveals gaps in your understanding. Make peer learning a priority.
- AI tools often lack your specific context and tend to provide generic responses. Your peers, who share your environment, can offer more relevant and accurate insights.
- Where AI tends to generate the most likely answer, your peers can provide alternative perspectives and valuable nuance. Rely on them as a quality checkpoint.

✓ 良い習慣：

私はAIに「並べ替え機能をテストするにはどうすればよいですか？」と尋ねます。AIはいくつかアイデアをくれます。私はそれらを試し、仲間とその結果をレビューします。私たちは一緒にアプローチを洗練させます。

✓ Good practice:

I ask AI: "How do I test a sorting function?" It gives me a few ideas. I try them out and review the results with a peer. We refine the approach together.

X 悪い習慣：

AIに関数全体を書かせ、それを自分のプロジェクトにコピー＆ペーストします。ピア評価の際、私はそれが何をするのか、なぜそうするのかを説明できません。私は信頼を失い、プロジェクトに失敗します。

X Bad practice:

I ask AI to write a whole function, copy-paste it into my project. During peer-evaluation, I can't explain what it does or why. I lose credibility and I fail my project.

✓ 良い習慣：

私はパーサーの設計にAIを使用します。その後、仲間とロジックを段階的に確認します。私たちは2つのバグを見つけ、より良く、よりきれいで、完全に理解できるものに一緒に書き直します。

✓ Good practice:

I use AI to help design a parser. Then I walk through the logic with a peer. We catch two bugs and rewrite it together better, cleaner, and fully understood.

X 悪い習慣：

私はCopilotにプロジェクトの重要な部分のコードを生成させます。コンパイルはできますが、パイプの処理方法を説明できません。評価中、私は正当化できず、プロジェクトに失敗します。

X Bad practice:

I let Copilot generate my code for a key part of my project. It compiles, but I can't explain how it handles pipes. During the evaluation, I fail to justify and I fail my project.

第3章

共通の指示

III.1

一般規則

Chapter III

Common Instructions

III.1

General Rules

- プロジェクトはPython 3.10以降で記述する必要があります。
- プロジェクトはflake8コーディング標準に準拠している必要があります。
- クラッシュを防ぐために、関数は例外を適切に処理する必要があります。潜在的なエラーを管理するためにtry-exceptブロックを使用してください。ファイルや接続などのリソースには、自動クリーンアップを確実にするためにコンテキストマネージャーを使用することを推奨します。レビュー中に処理されていない例外によりプログラムがクラッシュした場合、機能していないと見なされます。
- リークを防ぐために、すべてのリソース（ファイルハンドル、ネットワーク接続など）は適切に管理されなければなりません。自動処理のために可能な限りコンテキストマネージャーを使用してください。
- コードには、該当する場合（typingモジュールを使用）、関数パラメーター、戻り値の型、変数の型ヒントを含める必要があります。静的型チェックにはmypyを使用します。すべての関数はエラーなしでmypyを通過する必要があります。
- 目的、パラメーター、および戻り値を文書化するために、PEP 257（GoogleまたはNumPyスタイルなど）に従って、関数とクラスにdocstringを含めてください。

- Your project must be written in Python 3.10 or later.
- Your project must adhere to the flake8 coding standard.
- Your functions should handle exceptions gracefully to avoid crashes. Use try-except blocks to manage potential errors. Prefer context managers for resources like files or connections to ensure automatic cleanup. If your program crashes due to unhandled exceptions during the review, it will be considered non-functional.
- All resources (e.g., file handles, network connections) must be properly managed to prevent leaks. Use context managers where possible for automatic handling.
- Your code must include type hints for function parameters, return types, and variables where applicable (using the typing module). Use mypy for static type checking. All functions must pass mypy without errors.
- Include docstrings in functions and classes following PEP 257 (e.g., Google or NumPy style) to document purpose, parameters, and returns.

III.2

Makefile

III.2
Makefile

一般的なタスクを自動化するために、プロジェクトにMakefileを含めてください。これには以下のルールを含める必要があります（必須のlintは指定されたフラグを暗黙的に意味します。拡張チェックのためにtry-strictを使用することを強くお勧めします）。

- **install:** pip、uv、pipx、または任意のパッケージマネージャーを使用してプロジェクトの依存関係をインストールします。
- **run:** プロジェクトのメインスクリプトを実行します（例：Pythonインタープリター経由）。
- **debug:** Pythonの組み込みデバグガー（例：pdb）を使用してデバッグモードでメインスクリプトを実行します。
- **clean:** プロジェクト環境をクリーンに保つために、一時ファイルまたはキャッシュ（例：_pycache、.mypy_cache）を削除します。
- **lint:** flake8およびmypy --warn-return-any --warn-unused-ignores --ignore-missing-imports --disallow-untyped-defs --check-untyped-defsコマンドを実行します。
- **lint-strict（オプション）:** flake8およびmypy --strictコマンドを実行します。

Include a Makefile in your project to automate common tasks. It must contain the following rules (mandatory lint implies the specified flags; it is strongly recommended to try-strict for enhanced checking):

- **install:** Install project dependencies using pip, uv, pipx, or any other package manager of your choice.
- **run:** Execute the main script of your project (e.g., via Python interpreter).
- **debug:** Run the main script in debug mode using Python's built-in debugger (e.g., pdb).
- **clean:** Remove temporary files or caches (e.g., _pycache, .mypy_cache) to keep the project environment clean.
- **lint:** Execute the commands flake8 and mypy --warn-return-any --warn-unused-ignores --ignore-missing-imports--disallow-untyped-defs --check-untyped-defs
- **lint-strict (optional):** Execute the commands flake8. and mypy --strict

III.3

追加ガイドライン

III.3

Additional Guidelines

- プロジェクトの機能を検証するためのテストプログラムを作成してください（提出や採点はされません）。pytestやunittestなどのフレームワークを使用して、エッジケースをカバーするユニットテストを行います。
- Pythonのアーティファクトを除外するために.gitignoreファイルを含めてください。
- 開発中の依存関係の分離のために、仮想環境（venvやcondaなど）を使用することが推奨されます。

追加のプロジェクト固有の要件が適用される場合は、このセクションのすぐ下に記載されます。

- Create test programs to verify project functionality (not submitted or graded). Use frameworks like pytest or unittest for unit tests, covering edge cases.
 - Include a .gitignore file to exclude Python artifacts.
 - It is recommended to use virtual environments (e.g., venv or conda) for dependency isolation during development.
- If any additional project-specific requirements apply, they will be stated immediately below this section.

第4章

はじめに

```
# Chapter IV
# Introduction
```

自律型ドローンは輸送の未来です。それらはすでに、農業、建設、物流など、多くの産業で使用されています。また、監視や偵察などの軍事作戦でも使用されています。

あなたのタスクは、中央基地（開始位置）から目的地の場所（終了位置）へとドローン艦隊を効率的にルーティングし、一連の厳しい制約と最適化の目標の下でこの動的ネットワークをナビゲートするシステムを設計することです。

ゾーンのネットワークを表すグラフと、遵守しなければならない一連の制約が与えられます。グラフは接続されたゾーンのネットワークとして表され、接続はゾーン間の可能な移動経路を定義します。

Autonomous drones are the future of transportation. They are already used in many industries, such as agriculture, construction, and logistics. They are also used in military operations, such as surveillance and reconnaissance.

Your task is to design a system that efficiently routes a fleet of drones from a central base (start) to a target location (end), while navigating this dynamic network under a set of strict constraints and optimization goals.

You'll be given a graph representing the network of zones, and a set of constraints that you must respect.

The graph is represented as a network of connected zones, where connections define possible movement paths between zones.

第5章

制約事項

Chapter V Constraints

- グラフのロジックを支援するライブラリ（`networkx`、`graphlib`など）の使用は禁止されています。
 - プロジェクトは完全に型安全でなければなりません。flakesとmypyの使用は必須です。
 - プロジェクトは完全にオブジェクト指向でなければなりません。
- これはピアレビュー中に実証する必要があります。

- Any library that helps with graph logic is forbidden (such as `networkx`, `graphlib`, etc.).
 - The project must be completely typesafe. Using `flakes` and `mypy` is mandatory.
 - The project must be completely object-oriented.
- This will have to be demonstrated during the peer review.

第6章

ドローンを飛ばす

Chapter VI Let the drone fly

この課題には、ゾーンのネットワークを以下の形式で表す複数のファイルが添付されています。
例：

```
nb_drones: 5
start_hub: hub 0 0 [color=green]
end_hub: goal 10 10 [color=yellow]
hub: roof1 3 4 [zone=restricted color=red]
hub: roof2 6 2 [zone=normal color=blue]
hub: corridorA 4 3 [zone=priority color=green max_drones z=2]
hub: tunnelB 7 4 [zone=normal color=red]
hub: obstacleX 5 5 [zone=blocked color=gray]
connection: hub-roof1 1
connection: hub-corridorA
connection: roof1-roof2
connection: roof2-goal
connection: corridorA-tunnelB [max_link_capacity y=2]
connection: tunnelB-goal
```

Attached to this subject, you'll find multiple files that represent the network of zones in the following format:

Example:

```
nb_drones: 5
start_hub: hub 0 0 [color=green]
end_hub: goal 10 10 [color=yellow]
hub: roof1 3 4 [zone=restricted color=red]
hub: roof2 6 2 [zone=normal color=blue]
hub: corridorA 4 3 [zone=priority color=green max_drones z=2]
hub: tunnelB 7 4 [zone=normal color=red]
hub: obstacleX 5 5 [zone=blocked color=gray]
connection: hub-roof1 1
connection: hub-corridorA
connection: roof1-roof2
connection: roof2-goal
connection: corridorA-tunnelB [max_link_capacity y=2] connection: tunnelB-goal
```

面白いですね？より正確には：

- 最初の行は `nb_drones: <number>` を使用してドローンの数を定義します。
- 各行のゾーン定義はタイププレフィックスを使用します：
- `start_hub: <name> <x> <y> [metadata]` は開始ゾーンをマークします。
- `end_hub: <name> <x> <y> [metadata]` は終了ゾーンをマークします。
- `hub: <name> <x> <y> [metadata]` は通常のゾーンを定義します。
- 接続の構文では、ゾーン名にダッシュを使用することは禁止されています（以下を参照）。
- すべてのメタデータはオプションであり、デフォルト値とともにブラケット [...] で囲まれます：
- `zone=<type>`（デフォルト：normal）
- `color=<value>`（デフォルト：none）
- `max_drones=<number>`（デフォルト：1）このゾーンを同時に占有できるドローンの最大数
- ブラケット内のタグは任意の順序で表示できます。

Interesting, right? To be more precise:

- The first line defines the number of drones using `nb_drones: <number>`.
- Zone definition on each line using type prefixes:
- `start_hub: <name> <x> <y> [metadata]` marks the starting zone.
- `end_hub: <name> <x> <y> [metadata]` marks the end zone.
- `hub: <name> <x> <y> [metadata]` defines a regular zone.
- The connection syntax forbids dashes in zone names (see below).
- All metadata is optional and enclosed in brackets [...] with default values:
- `zone=<type>` (default: normal)
- `color=<value>` (default: none)
- `max_drones=<number>` (default: 1) Maximum drones that can occupy this zone simultaneously
- Tags inside brackets can appear in any order.

- ゾーンの種類：
- normal 1ターンの移動コストを持つ標準ゾーン（デフォルト）
- blocked - アクセスできないゾーン。ドローンはこのゾーンに入ったり通過したりしてはなりません。これを使用する経路は無効です。
- restricted 敏感または危険なゾーン。このゾーンへの移動には2ターンかかります。
- priority 優先されるゾーン。このゾーンへの移動には1ターンかかりますが、経路探索で優先されるべきです。
- 色：
- 色はオプションであり、視覚的表現（ターミナル出力やグラフィカル表示）に使用できます。
- colorに受け入れられる値は、有効な単一単語の文字列（例：red、blue、gray）です。許可される色の固定リストはありません。
- 色が指定された場合、実装は色付きのターミナル出力またはグラフィカルな表現を通じて視覚的なフィードバックを提供するべきです。

- Zone types:
- normal Standard zone with 1 turn movement cost (default)
- blocked - Inaccessible zone. Drones must not enter or pass through this zone. Any path using it is invalid.
- restricted A sensitive or dangerous zone. Movement to this zone costs 2 turns.
- priority A preferred zone. Movement to this zone costs 1 turn but should be prioritized in pathfinding.
- Colors:
- Colors are optional and can be used for visual representation (terminal output or graphical display).
- Accepted values for color are any valid single-word strings (e.g., red, blue, gray). There is no fixed list of allowed colors.
- When colors are specified, the implementation should provide visual feedback through colored terminal output or graphical representation.

- 接続は `connection: <name1>-<name2> [metadata]` を使用して定義されます：
 - 2つのゾーン間に双方向の接続（エッジ）を定義します。
 - 接続の構文では、ゾーン名にダッシュを使用することは禁止されています。
 - オプションのメタデータはブラケット [...] で指定できます：
 - * `max_link_capacity=<number>`（デフォルト：1）この接続を同時に横断できるドローンの最大数
 - コメントは '#' で始まり、無視されます。
- ゾーンの座標は常に整数であり、常に一意の開始ゾーンと一意の終了ゾーンが存在します。

- Connections are defined using `connection: <name1>-<name2> [metadata]`:
 - Define a bidirectional connection (edge) between two zones.
 - The connection syntax forbids dashes in zone names.
 - Optional metadata can be specified in brackets [...]:
 - * `max_link_capacity=<number>` (default: 1) Maximum drones that can traverse this connection simultaneously
 - Comments start with '#' and are ignored.
- The zones coordinates will always be integers, and there will always be a unique start and a unique end zone.

第7章

必須パート

Chapter VII

Mandatory Part

お察しの通り、主な目的は、すべてのドローンを開始ゾーンから終了ゾーンまで、可能な限り少ないシミュレーションターンで移動させることです。

As you may have guessed, the main objective is to move all drones from the start zone to the end zone in the fewest possible simulation turns.

VII.1

経路探索とアルゴリズムの要件

VII.1

Pathfinding and Algorithm Requirements

- ドローンは同時に移動することができます。アルゴリズムは、スループットを最大化し、不必要な遅延を避けるために経路をスケジューリングしなければなりません。
- 実装は以下を処理しなければなりません：
 - 複数の経路にわたるドローンの分配。
 - 移動が不可能な場合の戦略的な待機。
 - 経路の競合とデッドロックの回避。
- アルゴリズムは以下を考慮しなければなりません：
 - ゾーンのタイプ（例：restrictedやpriority）に関連する移動コストを含む経路の長さ。
 - ドローンが衝突したり互いをブロックしたりするのを防ぐためのターンのスケジューリング。
 - 利用可能な互いに素な経路または重複する経路を決定するためのグラフ構造。
 - ゾーンの容量制約（max_drones）と接続の容量（max_link_capacity）。
- あなたのアルゴリズムは適応性があるべきです。マップが異なれば、トポロジーやゾーンのタイプに応じて、異なるルーティング戦略が必要になる場合があります。

- Drones may move simultaneously. The algorithm must schedule paths to maximize throughput and avoid unnecessary delays.
- Your implementation must handle:
 - Distribution of drones across multiple paths.
 - Strategic waiting when movement is not possible.
 - Avoidance of path conflicts and deadlocks.
- The algorithm must take into account:
 - Path lengths, including movement costs associated with zone types (e.g., restricted or priority).
 - Turn scheduling, to prevent drones from colliding or blocking each other.
 - Graph structure, to determine available disjoint or overlapping paths.
 - Zone capacity constraints (max_drones) and connection capacity (max_link_capacity).
- Your algorithm should be adaptable: different maps may require different routing strategies, depending on the topology and zone types.

- 視覚的表現：実装は、以下のいずれかを通じてシミュレーションの視覚的フィードバックを提供しなければなりません：
 - ドローンの移動とゾーンの状態を示す色付きのターミナル出力
 - ネットワークとドローンの位置を表示するグラフィカルインターフェース
- ユーザー体験を向上させるための両方のオプション

- Visual Representation: Your implementation must provide visual feedback of the simulation, either through:
 - Colored terminal output showing drone movements and zone states
 - A graphical interface displaying the network and drone positions
- Both options for enhanced user experience

- あなたのアルゴリズムはどれくらい効率的ですか？
- 多数のドローンで機能しますか？
- 複雑さ（例： $O(n)$ 、 $O(\log n)$ など）はどのくらいですか？
- 経路を再計算していますか、それともキャッシュしていますか？
- メモリ使用量にどのように影響しますか？
- あなたの視覚的表現は、シミュレーションの理解をどのように深めますか？

- How efficient is your algorithm?
- Can it work with a large number of drones?
- What is the complexity (e.g., $O(n)$, $O(\log n)$, etc.)?
- Are you recalculating or caching paths?
- How does it impact memory usage?
- How does your visual representation enhance understanding of the simulation?

VII.2

ゾーンの占有規則

VII.2
Zone Occupancy Rules

- デフォルトでは、ゾーンには任意のシミュレーションターンに最大1台のドローンを含めることができます。
- max_drones-Nメタデータを持つゾーンは、同時に最大N台のドローンを含めることができます。
- 占有規則に対する特別な例外は以下の通りです：
- 開始ゾーン：すべてのドローンはここから始まり、最初はスペースを共有する場合があります。
- 終了ゾーン：複数のドローンがここに到着でき、配達されたと見なされます。
- ゾーンの容量が許さない限り、2台のドローンが同じターンに同じゾーンに入ることはできません。
- ドローンは、その最大容量を超えることになるゾーンに移動することはできません。
- 接続で定義された接続容量（max_link_capacity）は、同じ接続を同時に横断できるドローンの数を制限します。
- すべての容量制約が遵守されている限り、ドローンは同時に移動することができます。

- By default, a zone may contain at most one drone at any given simulation turn.
- Zones with max_drones-N metadata can contain up to N drones simultaneously.
- The only special exceptions to occupancy rules are:
- The start zone: all drones begin here and may share the space initially.
- The end zone: multiple drones can arrive here and are considered delivered.
- Two drones may not enter the same zone on the same turn unless the zone's capacity allows it.
- A drone may not move into a zone that would exceed its maximum capacity.
- Connection capacity (max_link_capacity) defined on connections limits how many drones can traverse the same connection simultaneously.
- Drones may move simultaneously, as long as all capacity constraints are respected.

VII.3

移動とターンの仕組み

VII.3

Movement and Turn Mechanics

シミュレーションは離散的なターンで進行します。各ターンで、すべてのドローンは以下を行うことができます：

- 隣接する接続されたゾーンに移動する（容量が許す場合）。
- restrictedゾーン（到達するのに2ターン必要なゾーン）に向かって接続に移動する。この場合、ドローンは次のターンの間に目的地に到達しなければなりません。接続上で余分なターンを待つことはできません。
- その場に留まる（待機するため、または移動がブロックされている場合など）。

シミュレーションは競合を防ぎ、ターンごとの状態評価に基づいて有効な移動スケジューリングを保証しなければなりません：

- ゾーンから移動するドローンは、その同じターンのために容量を解放します。
- ドローンが移動するためには、ゾーンに利用可能な容量がなければなりません（外に出るすべてのドローンがスペースを空けた後）。
- 複数ターンの移動（restrictedゾーン）の場合、ドローンは移動中に接続を占有し、指定されたターン数の後に目的地に到着しなければなりません。目的地ゾーンの空きスペースを接続上で待つことはできません。

The simulation proceeds in discrete turns. At each turn, every drone may:

- Move to an adjacent connected zone (if capacity allows).
- Move to a connection towards a restricted zone (that requires 2 turns to be reached). In this case, the drone MUST reach its destination during the next turn. It can't wait extra turns on the connection.
- Stay in place (e.g., to wait, or if movement is blocked).

The simulation must prevent conflicts and ensure valid movement scheduling based on turn-by-turn state evaluation:

- Drones moving out of a zone free up capacity for that same turn.
- A zone must have available capacity for a drone to move into it (after all drones moving out have freed up space).
- For multi-turn movements (restricted zones), the drone occupies the connection during transit and MUST arrive at the destination after the specified number of turns. It cannot wait on the connection for an empty space in the destination zone.

ゾーン間の各移動には、目的地のzone=typeに基づいて、ターン単位のコストがかかります：

- normal：1ターン（デフォルト）
- restricted：2ターン
- priority：1ターン（ただし、経路探索アルゴリズムで優先されるべきです）
- blocked：アクセス不可。入ることはできません

Each movement between zones has a cost in turns, based on the zone=type of the destination:

- normal: 1 turn (default)
- restricted: 2 turns
- priority: 1 turn (but should be preferred in pathfinding algorithms)
- blocked: Inaccessible cannot be entered

VII.4

パーサーの制約

VII.4

Parser Constraints

入力ファイルは、予想される構造と構文を遵守しなければなりません：

- 最初の行は `nb_drones: <positive_integer>` を使用してドローンの数を定義しなければなりません。
- プログラムは任意の数のドローン进行处理できなければなりません。
- `start_hub`: ゾーンと `end_hub`: ゾーンは正確に1つずつ存在しなければなりません。
- 各ゾーンには一意の名前と有効な整数の座標がなければなりません。
- ゾーン名は、ダッシュとスペースを除くすべての有効な文字を使用できます。
- 接続は、`connection: <zone1>-<zone2> [metadata]` を使用して、以前に定義されたゾーンのみをリンクしなければなりません。
- 同じ接続が2回以上出現してはなりません（例：a-b と b-a は重複と見なされます）。
- メタデータブロック（ゾーンの場合は `[zone=... color=...]`、接続の場合は `[max_link_capacity=...]` など）は、構文的に有効でなければなりません。
- ゾーンのタイプは、`normal`、`blocked`、`restricted`、`priority`のいずれかでなければなりません。無効なタイプはパーサーエラーを発生させる必要があります。
- 容量の値（ゾーンの`max_drones`、接続の`max_link_capacity`）は正の整数でなければなりません。
- `max_drones`容量は、`start_hub`および`end_hub`ゾーンでは無視されます。これらには容量制限がありません（すべてのドローンが開始ゾーンで開始でき、任意の数のドローンが終了ゾーンに配達される可能性があります）。これらの2つのゾーンにそのようなメタデータが存在する場合、それは無視され、検証エラーにはなりません。
- その他のパーサーエラーはプログラムを停止し、行と原因を示す明確なエラーメッセージを返さなければなりません。

The input file must respect the expected structure and syntax:

- The first line must define the number of drones using `nb_drones: <positive_integer>`.
- The program must be able to handle any number of drones.
- There must be exactly one `start_hub: zone` and one `end_hub: zone`.
- Each zone must have a unique name and valid integer coordinates.
- Zone names can use any valid characters except dashes and spaces.
- Connections must link only previously defined zones using `connection: <zone1>-<zone2> [metadata]`.
- The same connection must not appear more than once (e.g., a-b and b-a are considered duplicates).
- Any metadata block (e.g., `[zone=... color=...]` for zones, `[max_link_capacity=...]` for connections) must be syntactically valid.
- Zone types must be one of: `normal`, `blocked`, `restricted`, `priority`. Any invalid type must raise a parsing error.
- Capacity values (`max_drones` for zones, `max_link_capacity` for connections) must be positive integers.
- The `max_drones` capacity is ignored on the `start_hub` and `end_hub` zones: these have no capacity limit (all drones may start in the start zone, and any number of drones may be delivered to the end zone). If such metadata is present on those two zones, it is ignored and is

not a validation error.

- Any other parsing error must stop the program and return a clear error message indicating the line and cause.

エッジケースやエラー処理を処理するために、課題で提供されているマップファイルの上に独自のマップファイルを作成することを強くお勧めします。

It's highly recommended to make your own map files on top of the ones provided in the subject for handling edge cases and error handling.

VII.5

シミュレーション出力形式

VII.5

Simulation Output Format

- シミュレーションは、開始ゾーンから終了ゾーンへのドローンのステップバイステップの移動を出力しなければなりません。
- 各シミュレーションターンは1行で表されます。
- 1行には、そのターンに発生するすべてのドローンの移動をスペース区切りでリストしなければなりません。各移動は、フォーマット：D<ID>-<zone>、またはrestrictedゾーンに向かって飛行中のドローンの場合は D<ID>-<connection> に従わなければなりません。
- D<ID> は一意のドローン識別子（例：D1、D2）を指します。
- <zone> は目的地ゾーンの名前です。
- <connection> はrestrictedゾーンに向かう接続の名前です。
- 特定のターンで移動しないドローンは、その行から省略されます。
- 終了ゾーンに到達したドローンは配達されたと見なされ、もはや追跡されません。
- すべてのドローンが終了ゾーンに到達した時点でシミュレーションは終了します。
- 例：

D1-roof1 D2-corridorA

D1-roof2 D2-tunnelB

D1-goal D2-goal

- The simulation must output the step-by-step movement of drones from the start to the end zone.
- Each simulation turn is represented by a line.
- A line must list all the drone movements that occur during that turn, space-separated. Each movement must follow the format: D<ID>-<zone>, or D<ID>-<connection> in case of drones still in flight toward restricted zones.
- D<ID> refers to the unique drone identifier (e.g., D1, D2).
- <zone> is the name of the destination zone.
- <connection> is the name of the connection toward a restricted zone.
- Drones that do not move in a given turn are omitted from that line.
- Drones that reach the end zone are considered delivered and are no longer tracked.
- The simulation ends when all drones have reached the end zone.

• Example:

D1-roof1 D2-corridorA

D1-roof2 D2-tunnelB

D1-goal D2-goal

VII.6

スコアリングシステム

VII.6

Scoring System

- ソリューションのパフォーマンスは、すべてのドローンを開始ゾーンから終了ゾーンにルーティングするために必要なシミュレーションターンの総数に基づいて評価されます。
- ターン数が少ないほど、スコアが高くなります。
- 有効なシミュレーションは以下を行わなければなりません：
- すべての移動および占有規則に準拠する。
- ゾーンタイプに関連する移動コストを正しく処理する。
- すべての容量制約（ゾーンおよび接続の制限）を遵守する。
- すべての競合を回避する（例：ゾーンまたは接続の容量を超えること）。

- The performance of a solution is evaluated based on the total number of simulation turns required to route all drones from the start zone to the end zone.
- The fewer the number of turns, the better the score.
- A valid simulation must:
- Comply with all movement and occupancy rules.
- Correctly handle movement costs associated with zone types.
- Respect all capacity constraints (zone and connection limits).
- Avoid all conflicts (e.g., exceeding zone or connection capacity).

二次的な（オプションの）評価指標には以下が含まれる場合があります：

- ターンごとに移動するドローンの数（経路割り当ての効率）。
- ドローンあたりの平均ターン数。
- 総経路コスト（すべてのドローンにわたる加重移動コストの合計）。
- 視覚的表現の品質と有用性。

ターン数が同一の場合、ソリューションは二次指標またはコードの品質に基づいて比較されることがあります。

Secondary (optional) evaluation metrics may include:

- The number of drones moved per turn (efficiency of path allocation).
- The average number of turns per drone.
- The total path cost (sum of weighted movement costs across all drones).
- Quality and usefulness of visual representation.

In case of identical turn counts, solutions may be compared based on secondary metrics or code quality.

これらの二次指標は自動的に計算することは必須ではありませんが、学習者は仲間のパフォーマンス評価を支援するために、シミュレーション出力またはドキュメントにそれらを表示することが推奨されます。

These secondary metrics are not mandatory to compute automatically, but learners are encouraged to display them in their simulation output or documentation to help peers evaluate performance.

VII.7

パフォーマンスベンチマーク

VII.7

Performance Benchmarks

以下のパフォーマンス目標は、実装が達成しなければならない予想される最適化レベルを定義しています。

- 予想されるパフォーマンス：
- 簡単なマップは10ターン未満で解決されるべきです
- 中程度のマップは10～30ターンで解決されるべきです
- 難しいマップは60ターン未満で解決されるべきです
- チャレンジャーマップ（オプション）は、45ターンの参考記録を破ることを目指すべきです。このレベルは純粋にオプションであり、成績には影響しません。

The following performance targets define the expected optimization level your implementation must achieve.

- Expected performance:
- Easy maps should be solved in less than 10 turns
- Medium maps should be solved in 10-30 turns
- Hard maps should be solved in less than 60 turns
- Challenger map (optional) should aim to beat the reference record of 45 turns This level is purely optional and does not affect your grade.

アルゴリズムの効率を評価するのに役立つように、提供されたテストマップに基づく参考パフォーマンス目標を以下に示します：

- 簡単なマップ：
 - 2台のドローンによる直線経路：目標 ≤ 6 ターン
 - 4台のドローンによる単純な分岐：目標 ≤ 8 ターン
 - 4台のドローンによる基本容量：目標 ≤ 6 ターン
- 中程度のマップ：
 - 5台のドローンによる行き止まりの罠：目標 ≤ 12 ターン
 - 6台のドローンによる円形ループ：目標 < 15 ターン
 - 5台のドローンによる優先順位パズル：目標 ≤ 12 ターン
- 難しいマップ：
 - 8台のドローンによる迷路の悪夢：目標 ≤ 30 ターン
 - 12台のドローンによる容量の地獄：目標 ≤ 35 ターン
 - 15台のドローンによる究極の挑戦：目標 ≤ 45 ターン
- チャレンジャーマップ（並外れた実装のためのオプション）：
 - 25台のドローンによる不可能な夢：参考記録：45ターン
 - このほぼ解決不可能な課題は、アルゴリズムの研究と最適化のために設計されています
 - このマップを解決することは、並外れた経路探索と最適化のスキルを実証します
 - 注：このレベルは純粋にオプションであり、成績には影響しません

To help you evaluate your algorithm's efficiency, here are reference performance targets based on the provided test maps:

- Easy Maps:
 - Linear path with 2 drones: Target ≤ 6 turns
 - Simple fork with 4 drones: Target ≤ 8 turns
 - Basic capacity with 4 drones: Target ≤ 6 turns
- # • Medium Maps:
 - Dead end trap with 5 drones: Target ≤ 12 turns
 - Circular loop with 6 drones: Target < 15 turns
 - Priority puzzle with 5 drones: Target ≤ 12 turns
- # • Hard Maps:
 - Maze nightmare with 8 drones: Target ≤ 30 turns
 - Capacity hell with 12 drones: Target ≤ 35 turns
 - Ultimate challenge with 15 drones: Target ≤ 45 turns
 - Challenger Map (optional for exceptional implementations):
 - The Impossible Dream with 25 drones: Reference record: 45 turns
 - This quasi-unsolvable challenge is designed for algorithmic research and optimization
 - Solving this map demonstrates exceptional pathfinding and optimization skills
 - Note: This level is purely optional and does not affect your grade

これらのベンチマークは、アルゴリズムのパフォーマンスを評価するのに役立つ最適化目標として提供されています。これらの目標を達成することは、十分に最適化された実装を実証し、ピア評価の際に評価されます。

- あなたのアルゴリズムはこれらのパフォーマンスベンチマークを満たすことができますか？
- あなたのソリューションは参考目標と比べてどうですか？
- パフォーマンスを向上させるためにどのような最適化を実装しましたか？
- チャレンジャーマップを解決し、45ターンの記録を破ることはできますか？

These benchmarks are provided as optimization targets to help you evaluate your algorithm's performance. Meeting these targets demonstrates a well-optimized implementation and will be assessed during peer evaluation.

- Can your algorithm meet these performance benchmarks?
- How does your solution compare to the reference targets?
- What optimizations did you implement to achieve better performance?
- Can you solve the Challenger map and beat the 45-turn record?

開始 P: D1 3 PD2

図 VII.1: マップ easy 2

```
start
P: D1 3
PD2
Figure VII.1: Map easy 2
```

開始 fast_junction fast D3 1th merge D4 point slow_path 1 slow D5

図 VII.2: マップ medium 3

```
start
fast_junctio n
fas D3 1th
mer D4 point
slow_path 1
slov Dsath D5
Figure VII.2: Map medium 3 18 goal
```

priority bypass priority bypass

図 VII.3: マップ hard 2

convergence

Drones are interesting.

ority bypa

ority bypa

Figure VII.3: Map hard 2 19 convergence

Drones are interesting.

第8章

Readmeの要件

Chapter VIII

Readme Requirements

GitリポジトリのルートにREADME.mdファイルを提供する必要があります。その目的は、プロジェクトに不慣れな人（同僚、スタッフ、採用担当者など）が、プロジェクトの概要、実行方法、トピックに関する詳細情報の場所をすばやく理解できるようにすることです。

A README.md file must be provided at the root of your Git repository. Its purpose is to allow anyone unfamiliar with the project (peers, staff, recruiters, etc.) to quickly understand what the project is about, how to run it, and where to find more information on the topic.

README.mdには少なくとも以下を含める必要があります：

- 最初の行は斜体で、「This project has been created as part of the 42 curriculum by <login1>[, <login2>[, <login3>[...]]].」と記述する必要があります。
- 目標と簡単な概要を含む、プロジェクトを明確に示す「説明」セクション。
- コンパイル、インストール、および/または実行に関する関連情報を含む「指示」セクション。
- トピックに関連する古典的な参考資料（ドキュメント、記事、チュートリアルなど）をリストした「リソース」セクション、およびAIがどのように使用されたか（どのタスクに、プロジェクトのどの部分に）の記述。

プロジェクトによっては、追加のセクション（使用例、機能リスト、技術的選択など）が必要になる場合があります。

必要な追加事項は、以下に明示的にリストされます。

- アルゴリズムの選択と実装戦略の詳細な説明も含める必要があります。
- 視覚的表現機能と、それらがユーザー体験をどのように向上させるかについてのドキュメント。
- プログラムの機能を実証する入力例と予想される出力。

READMEは英語で書かれなければなりません。

The README.md must include at least:

- The very first line must be italicized and read: This project has been created as part of the 42 curriculum by <login1>[, <login2>[, <login3>[...]]].
- A "Description" section that clearly presents the project, including its goal and a brief overview.
- An "Instructions" section containing any relevant information about compilation, installation, and/or execution.
- A "Resources" section listing classic references related to the topic (documentation, articles, tutorials, etc.), as well as a description of how AI was used specifying for which tasks and which parts of the project.

Additional sections may be required depending on the project (e.g., usage examples, feature list, technical choices, etc.).

Any required additions will be explicitly listed below.

- A detailed description of your algorithm choices and implementation strategy must also be included.
- Documentation of the visual representation features and how they enhance the user experience.
- Example input and expected output demonstrating the program's functionality.

Your README must be written in English.

第9章

ボーナスパート

Chapter IX

Bonus Part

このボーナスパートは、すべての必須要件が満たされている場合にのみレビューされます。プロジェクトを強化するために実装できる機能は以下の通りです：

- 並外れたパフォーマンス：
- 提供されたすべてのマップのパフォーマンス参考目標を「完璧に」満たしている。
 - 「完璧に」とは、目標のターン数に一致するか、それを破ることを意味します。
- チャレンジャーマップ：
- The Impossible Dreamマップが解決され、45ターンの参考記録を破る。

This Bonus part will be reviewed only if all the mandatory requirements are met. Here are features you can implement to enhance your project:

- Exceptional performance:
- You 'perfectly' meet the performance reference targets for all provided maps.
 - 'perfectly' means you match or beat the target turn count.
- Challenger map:
- The Impossible Dream map is solved and beats the reference record of 45 turns.

第10章

提出とピアレビュー

Chapter X

Submission and peer-review

通常通り、Gitリポジトリに課題を提出してください。ピア評価中に評価されるのは、リポジトリ内の作業のみです。ファイル名が正しいことを確認するために、ためらわずに再確認してください。

すべてのファイルをリポジトリのルートに配置してください。

Submit your assignment in your Git repository as usual. Only the work inside your repository will be evaluated during the peer-evaluation. Don't hesitate to double-check the names of your files to ensure they are correct.

Place all your files at the root of your repository.

以下を含む、Pythonで書かれた完全に機能するシミュレーション：

- 入力ファイルフォーマットのパーサー。
- 移動とゾーンのルールを尊重するシミュレーションエンジン。
- 総ターン数を最小化できる経路探索アルゴリズム（複数可）。
- 視覚的表現システム（ターミナルカラーおよび/またはグラフィカルインターフェース）。
- 指定されたフォーマットに従うターミナルまたはログ出力。

A fully working simulation written in Python, including:

- A parser for the input file format.
 - A simulation engine respecting movement and zone rules.
 - A pathfinding algorithm (or multiple) capable of minimizing total turns.
 - A visual representation system (terminal colors and/or graphical interface).
 - A terminal or log output that follows the specified format.
-

コードを説明するよう求められたり、場合によってはコードを書くよう求められたりすることがあることに注意してください。これに対する準備をしておいてください。

評価マップは、課題で提供されたものとは異なる場合があります。

評価中、プロジェクトの簡単な変更が時折要求されることがあります。これには、軽微な動作の変更、書き換える数行のコード、または追加しやすい機能が含まれる可能性があります。

この手順はすべてのプロジェクトに適用されるわけではありませんが、評価ガイドラインに記載されている場合は、その準備をしておく必要があります。

この手順は、プロジェクトの特定の部分に関するあなたの実際の理解を検証することを目的としています。変更は、選択した任意の開発環境（例：通常のセットアップ）で実行でき、評価の一部として特定の時間枠が定義されていない限り、数分以内に実行可能であるべきです。

たとえば、関数やスクリプトの小さな更新を行ったり、表示を変更したり、新しい情報を保存するためにデータ構造を調整したりするよう求められる場合があります。

詳細（範囲、ターゲットなど）は評価ガイドラインに指定されており、同じプロジェクトでも評価ごとに異なる場合があります。

Note that we may ask you to explain your code or possibly even to write some code. Make sure to be prepared for this.

Evaluation maps may be different from the ones provided in the subject.

During the evaluation, a brief modification of the project may occasionally be requested. This could involve a minor behaviour change, a few lines of code to write or rewrite, or an easy-to-add feature.

While this step may not be applicable to every project, you must be prepared for it if it is mentioned in the evaluation guidelines.

This step is meant to verify your actual understanding of a specific part of the project. The modification can be performed in any development environment you choose (e.g., your usual setup), and it should be feasible within a few minutes unless a specific time frame is defined as part of the evaluation.

You can, for example, be asked to make a small update to a function or script, modify a display, or adjust a data structure to store new information, etc.

The details (scope, target, etc.) will be specified in the evaluation guidelines and may vary from one evaluation to another for the same project.